

Process Documentation

Julian Mora

WEEK 1

```
let dim = 400
let num = 10
let reduction = dim/num
let posX=0
let posY=0
```

```
function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255)
  frameRate(10)
  rectMode(CENTER)
  angleMode(DEGREES)
  posX = width /2
  posY = height * 0.5
```

```
}
```

```
function draw() {
  background(255)
  noFill()
  strokeWeight(1)
  stroke(0)
  dim=150 + (sin(frameCount) *50)
  num=20
```

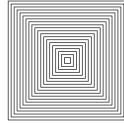
```
  reduction = dim/num
```

```
// for loop
```

```
for(let i=0; i<num; i++){
  square(posX, posY, (dim) - (reduction * i))
```

```
}
```

```
}
```



```
let dimX = 453
let dimY = dimX
let num = 17
let reduction = dimX / num
let posX = 0
let posY = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255)
  rectMode(CENTER)
  angleMode(DEGREES)
  posX = width / 2
  posY = height / 2
}

function draw() {
  background(255)
  noFill()
  strokeWeight(3)
  stroke(0)
  let numX = 10 //number of squares

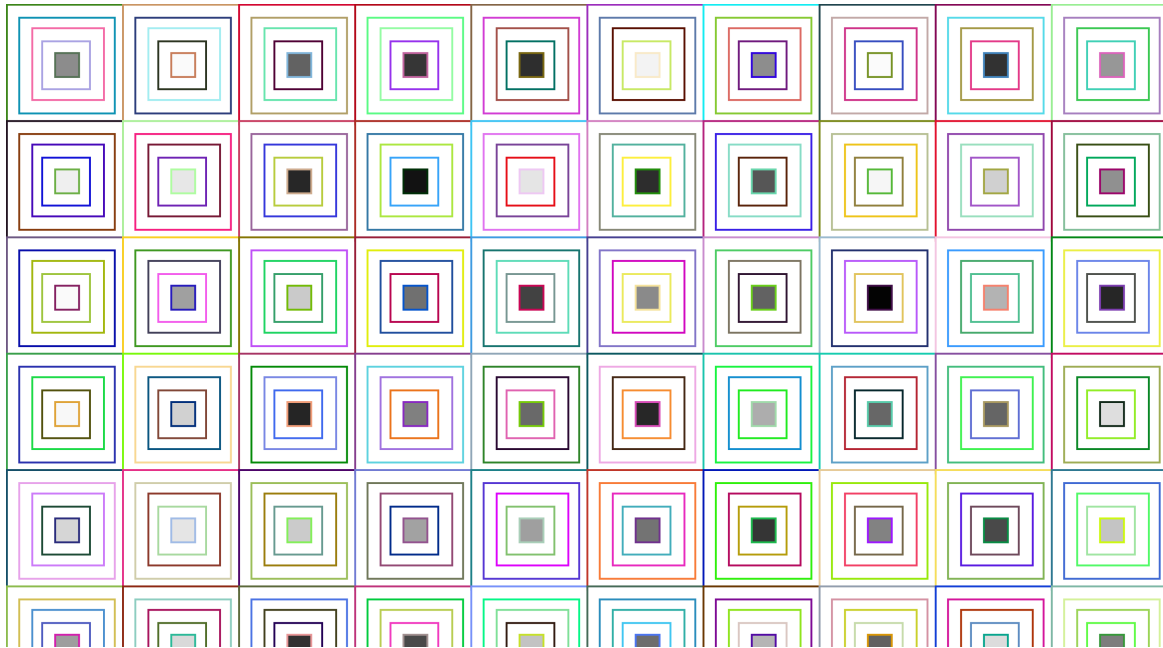
  //forloop
  for(let i = 0; i < numX; i++) { //here we use i variable because it is numX
    let dimension = width / numX //define dimension
    let posX = dimension / 2 + (i * dimension)
    for(let j = 0; j < numX; j++) { //here we use j variable because it is numY the
      number here defines the ammount of rows, we use numX
      let posY = dimension / 2 + (j * dimension)
      tmcs(posX, posY, 4, dimension, 5) //5 is the num parameter, if i affect it here it
      affects the ammount of squares
    }
  }
}
```

```
}
```

```
// this function draws squares at position
// x and y, and they move randomly
// a tiny bit
function tmcs(x, y, speed, dim, num) {

  let dimension = dim + sin(frameCount * speed) * 10
  // num = 10
  let reduction = dimension / num

  for(let i = 0; i < num; i++) {
    let offsetX = noise(frameCount * 0.00001, frameCount * 0.00003, i) * 10
    let offsetY = noise(frameCount * 0.00001, frameCount * 0.00003, i) * 10
    let r = random(255)
    let g = random(255)
    let b = random(255)
    fill(255) //fill the little square with color
    if(i == num-1) { //the center square is defined by num-1 for the fill
      fill(random(255))
    }
    stroke(r, g, b)
    strokeWeight(3)
    square(
      x + offsetX,
      y + offsetY,
      (dimension) - (reduction * i)
    )
  }
}
```



```
// Julian Mora - Week 1 variation: Radial concentric circles
// inspired by Vera Molnar, from squares to circles in polar grid
```

```
let rings = 8;
let cols = 12;

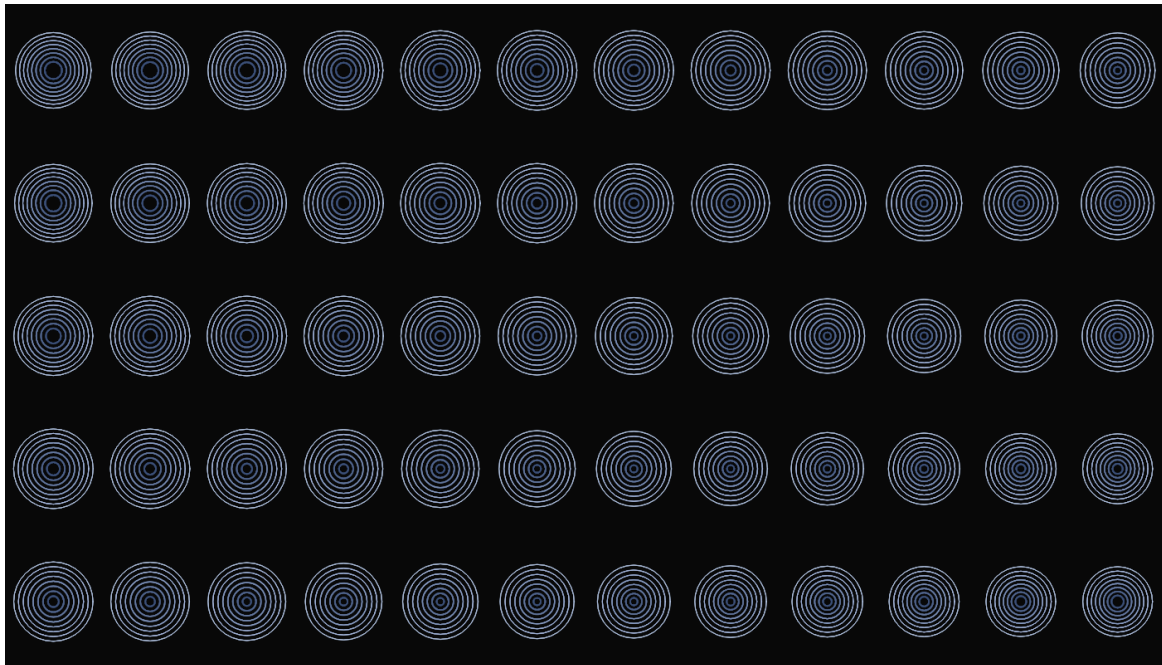
function setup() {
  createCanvas(windowWidth, windowHeight);
  background(10);
  angleMode(DEGREES);
  noFill();
}
```

```

}

function draw() {
  background(10, 30);
  let t = frameCount * 0.4;
  for (let col = 0; col < cols; col++) {
    let cx = (width / cols) * col + (width / cols) / 2;
    for (let row = 0; row < 5; row++) {
      let cy = (height / 5) * row + (height / 5) / 2;
      let phase = col * 15 + row * 25;
      for (let r = 0; r < rings; r++) {
        let d = map(r, 0, rings, 90, 4);
        let wobble = sin(t + phase + r * 12) * 6;
        let br = map(r, 0, rings, 200, 40);
        stroke(br, br + 20, br + 60);
        strokeWeight(map(r, 0, rings, 0.5, 2));
        ellipse(cx, cy, d + wobble, d + wobble);
      }
    }
  }
}

```



WEEK 2

```

/*
_HY5_p5_hydra // cc teddavis.org 2024
pass p5 into hydra
docs: https://github.com/ffd8/hy5
*/

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

osc(27, 0.3, 3.9).kaleid(4).out(o0)
noise(4).pixelate(27, 12).out(o1)

```

```

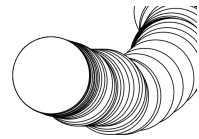
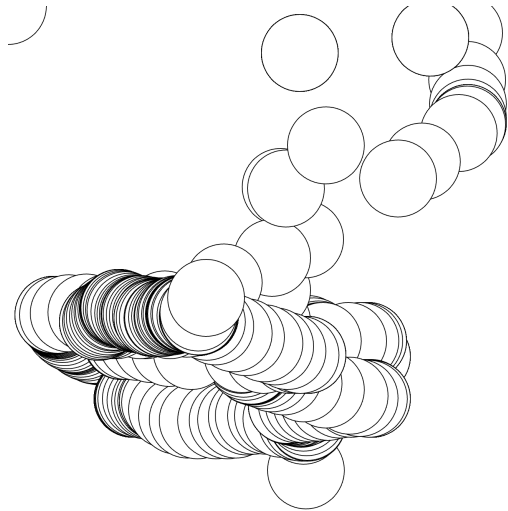
render(o0)    // show stripes
render(o)     // show noise

// sandbox - end

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  // clear()
  circle(mouseX, mouseY, 100)
}

```



```

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  background(230);
  fill(10);
  textSize(100)
  textWrap(WORD)
  textFont("american typewriter")
  textAlign(CENTER)
  textLeading(240)
  text("NACHBAR WIE GEHTS DIR HEUTE? GESTERN? MORGEN?", 0, 400, windowWidth,
  windowHeight)
}

```

NACHBAR WIE GEHTS DIR HEUTE? GESTERN?

```
function setup() {  
  createCanvas(windowWidth, windowHeight)  
  
}  
  
function draw() {  
  let live = frameCount%8;  
  let sine = floor(5*sin(frameCount/10)+5)  
  let words = ["nostalgia", "drops", "falling","into", "everything", "mankind",  
  "has", "created"]  
  let rand = random(words);  
  
  frameRate(5)  
  background(230, 100, 20)  
  fill(10)  
  textSize(100)  
  textWrap(CHAR)  
  textFont('american typewriter')  
  textAlign(CENTER)  
  textStyle(NORMAL)  
  textLeading(40)  
  text(words[live].replace(/[o]/g, "x").repeat(100),  
  80, 100, windowWidth/1.1, windowHeight);  
  
}
```



WEEK 3 Mid-term Presentations

// Julian Mora – Week 4 mid-term piece: binary rain
// matrix wall of 0s and 1s – reacts to mouse position

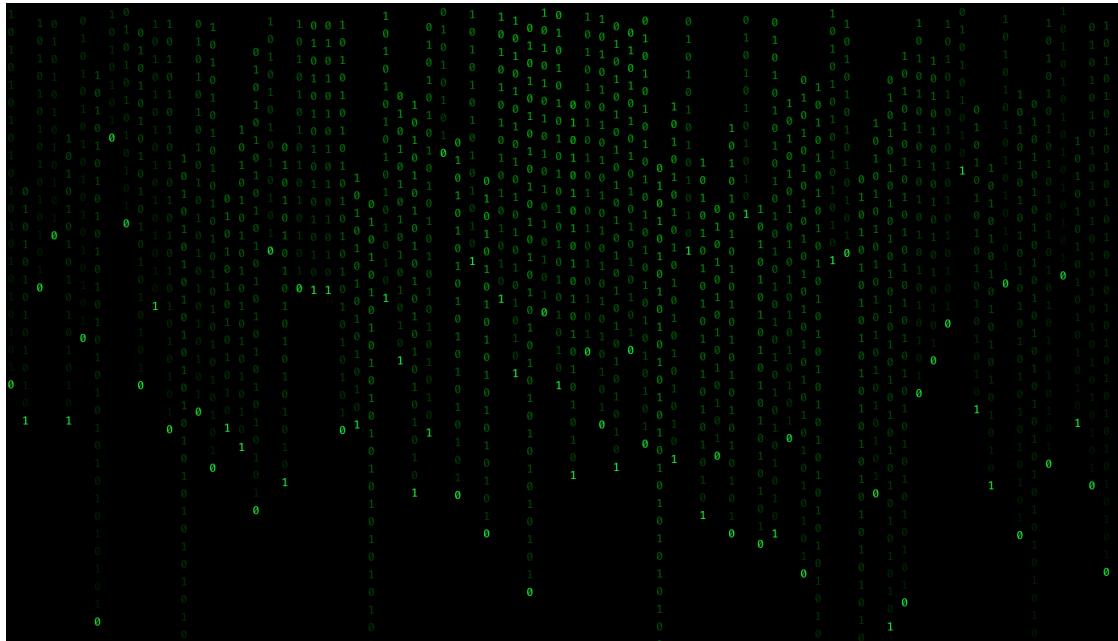
```
let cols;  
let streams = [];  
let cellW = 18;  
let cellH = 22;  
  
function setup() {  
  createCanvas(windowWidth, windowHeight);  
  textFont('monospace');  
  textSize(14);  
  cols = floor(width / cellW);  
  for (let i = 0; i < cols; i++) {  
    streams.push({  
      y: random(-height, 0),  
      speed: random(1.5, 4),  
      len: floor(random(12, 30)),  
    });  
  }  
}  
  
function draw() {  
  background(0, 0, 0, 50);  
  let mx = mouseX / width;  
  for (let i = 0; i < streams.length; i++) {  
    let s = streams[i];  
    let x = i * cellW;  
    for (let j = 0; j < s.len; j++) {  
      let y = s.y - j * cellH;  
      if (y < 0 || y > height) continue;  
      let dist = abs((x / width) - mx);  
      let bright = map(dist, 0, 0.5, 255, 40);  
      let alpha = map(j, 0, s.len, 255, 30);  
      let bit = (frameCount + i * 7 + j * 3) % 2;  
      fill(j == 0 ? bright : 20, j == 0 ? 255 : bright * 0.55, 40, alpha);  
      noStroke();  
      text(bit, x, y);  
    }  
    s.y += s.speed;  
    if (s.y - s.len * cellH > height) {  

```

```

    s.y = random(-200, 0);
    s.speed = random(1.5, 4);
  }
}

```



```

// Julian Mora - Week 4 mid-term piece: CREATIVE CODING breathing text
// 80 characters orbiting the center, breathing with sin() modulation

```

```

let chars = "CREATIVE CODING";
let n = 80;

function setup() {
  createCanvas(windowWidth, windowHeight);
  textAlign(CENTER, CENTER);
  angleMode(DEGREES);
}

function draw() {
  background(0, 18);
  let t = frameCount * 0.5;
  let cx = width / 2;
  let cy = height / 2;
  for (let i = 0; i < n; i++) {
    let angle = (i / n) * 360 + t * 0.3;
    let breath = sin(t * 1.2 + i * 6) * 55 + 210;
    let x = cx + cos(angle) * breath;
    let y = cy + sin(angle) * breath * 0.52;
    let ch = chars[i % chars.length];
    let sz = map(sin(t * 0.8 + i * 4.5), -1, 1, 18, 48);
    let br = map(sin(t * 1.4 + i * 5), -1, 1, 60, 255);
    textFont('monospace');
    textSize(sz);
    fill(br, br * 0.82, br);
    noStroke();
    text(ch, x, y);
  }
  // inner ring
  for (let i = 0; i < 40; i++) {
    let angle = (i / 40) * 360 - t * 0.5;
    let r = sin(t * 0.9 + i * 9) * 22 + 95;
    let x = cx + cos(angle) * r;
    let y = cy + sin(angle) * r * 0.55;
    let br = map(sin(t * 1.2 + i * 6), -1, 1, 60, 180);
  }
}

```

```
    textFont('monospace');
    textSize(16);
    fill(br, br, br * 0.88, 180);
    text(chars[(i + 3) % chars.length], x, y);
  }
}
```

Week 1

Week 1 was about learning to think in loops. The molar animation — concentric squares whose size breathes with `sin()` — was the first sketch where I felt something alive on the screen. Day 2 I extracted the logic into a function (`tmcs`) and duplicated it across a 10×5 grid. Then I replaced the squares with concentric circles arranged in a polar grid, which made the same nested-loop structure feel completely different just by changing one shape call.

Week 2

Week 2 was one external workshop after another, each showing a completely different way code can behave. Hydra was the first tool where there was no `draw()` — the synthesis loop runs itself. The concrete poetry session with Jasmin Meerhoff was the one that stayed: treating text as material, each word a tile you can tile, mutate, repeat. The words animation used the same regex idea — `replace()`, `repeat()` — to make language into texture.

Week 3 + Mid-term Presentations

For the mid-term my colleague Antonio and I prepared two visual pieces: a binary rain — a wall of 0s and 1s falling in columns, brightness reacting to the mouse position — and a breathing text orbit where 80 characters from "CREATIVE CODING" circle the canvas with each one pulsing at a different `sin()` phase. The binary rain came from the matrix-code reference Antonio had in mind.